
ÁRBOLES BINARIOS DE BÚSQUEDA BALANCEADO

— 23-Abril-2026 —

Altura de un árbol

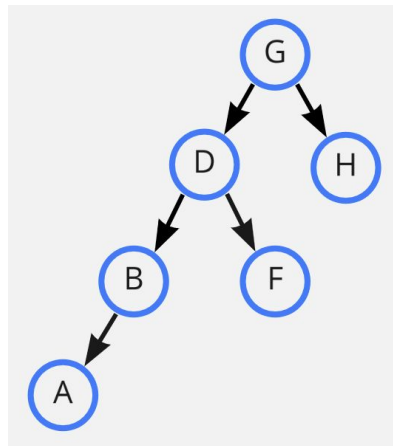
La altura de un nodo v , denotado por $h(v)$, se define de la siguiente forma:

- Si $v = null$ $h(v) = -1$
- Sean v_i y v_d los hijos de v $h(v) = 1 + \max(h(v_i), h(v_d))$

Árbol Binario de Búsqueda Balanceado

Un árbol binario de búsqueda T es c -balanceado si y sólo si para todo nodo de T se cumple que la distancia máxima del nodo a alguna de sus hojas es a lo más c veces la distancia mínima del vértice a alguna de sus hojas, donde c es una constante fija.

Este árbol es 3-balanceado.



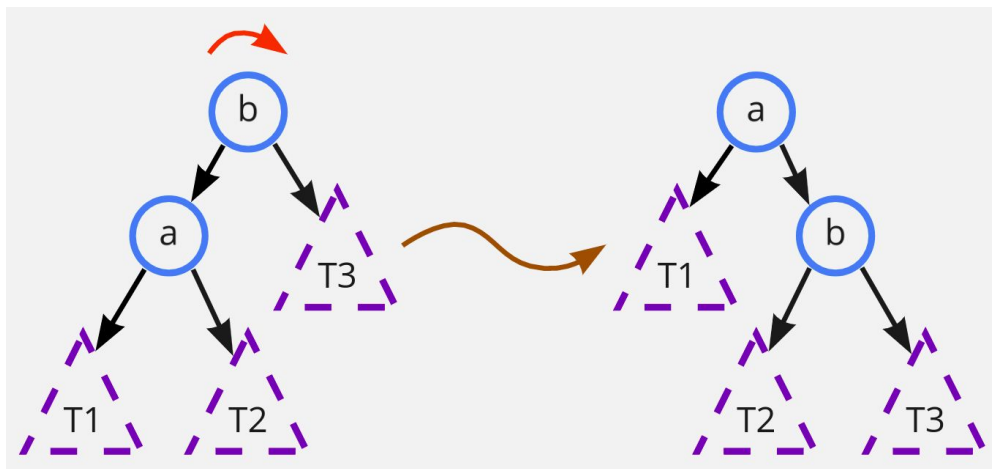
Rotaciones

Para balancear un BST hay que reacomodarlo, por lo que se tienen funciones de rotación.

Un nodo se puede rotar hacia la derecha o hacia la izquierda, el único requisito es que tenga un hijo en la dirección opuesta a la que se va a rotar.

Rotar a la derecha

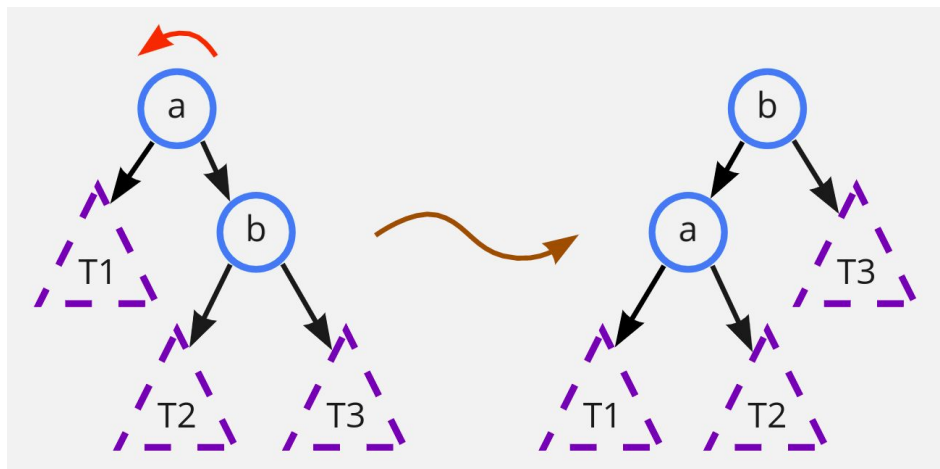
Al rotar a la derecha un nodo, se convierte en el hijo derecho del que antes era su hijo izquierdo y el subárbol derecho del que antes era su hijo, se vuelve su subárbol izquierdo.



Nota que el orden del BST se preserva.

Rotar a la izquierda

Al rotar a la izquierda un nodo, se convierte en el hijo izquierdo del que antes era su hijo derecho y el subárbol izquierdo del que antes era su hijo, se vuelve su subárbol derecho.



Nota que el orden del BST también se preserva.

Altura

Recordemos cómo habíamos definido la altura. Sea v un nodo de un árbol binario, la altura del nodo se denota $h(v)$ donde:

- Si v es *null*, entonces $h(v) = -1$
- Sean v_i y v_d los hijos de v , entonces $h(v) = 1 + \max(h(v_i), h(v_d))$

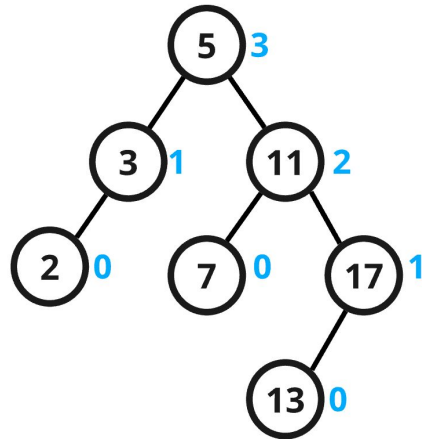
Al programarlo, el cálculo de las alturas va de las hojas a la raíz.



Árboles AVL - 1962 Adelson-Velsky y Evgenii Landis

Un árbol binario de búsqueda **T** es AVL si cumple la siguiente propiedad:

- Para todo nodo no vacío **v** $\in$ **T**, siendo **hi** y **hd** sus hijos, se cumple que $|h(\mathbf{hi}) - h(\mathbf{hd})| < 2$.



● Altura

- ★ Al programar árboles AVL es conveniente que cada nodo guarde su altura.

Balance de Árboles AVL

Para facilitar el entendimiento del balance de vértices en un árbol AVL, podemos definir la función **b(v)**:

- **b(v) = h(hi) - h(hd)**, donde **hi** y **hd** son los hijos izquierdo y derecho respectivamente de **v**.

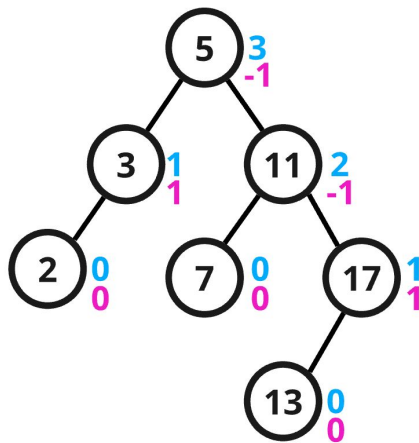


Balance de Árboles AVL

Entonces, en un árbol AVL, para todo nodo no vacío $v \in T$, se cumple que $|b(v)| < 2$.

Dicho de otro modo, $-1 \leq b(v) \leq 1$.

● **Altura**
● **Balance**



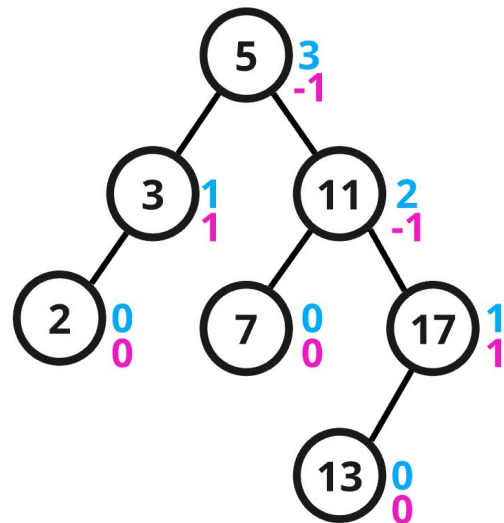
★ Al programarlo no es necesario que cada nodo guarde el valor de su balance.

Búsqueda en Árboles AVL

Como un árbol AVL es un BST, el algoritmo de búsqueda sigue siendo el mismo que en un BST cualquiera.

La condición de un AVL obliga que si la longitud de la trayectoria mínima de un vértice cualquiera a alguna de sus hojas es k , entonces la longitud de trayectoria máxima será a lo más $k + 1$.

Así que podemos considerar a los árboles AVL balanceados, por lo que en la búsqueda $T(n) \in O(\log n)$.



Inserción en Árboles AVL

Al agregar un nuevo nodo en un árbol AVL puede ocurrir que la altura de algunos nodos aumente provocando un desbalance , así que hay que actualizar las alturas y rebalancear después de insertar un nodo.

Algoritmo para agregar un elemento **e** a un árbol AVL:

- 1 Insertamos **e** al árbol como un BST cualquiera.
- 2 Sea **v** el nodo donde se guarda **e**, actualizamos alturas y rebalanceamos desde **v** hasta la raíz.

Eliminación en Árboles AVL

Al remover un nodo de un árbol AVL puede ocurrir que la altura de algunos nodos disminuya provocando un desbalance , así que hay que actualizar las alturas y rebalancear después de eliminar un nodo.

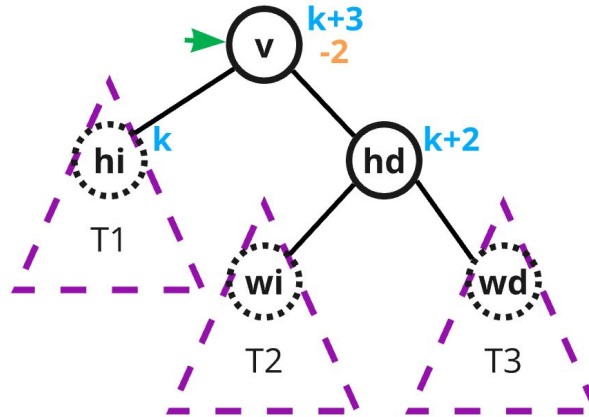
Algoritmo para eliminar un elemento **e** de un árbol AVL:

- 1 Eliminamos **e** del árbol como un BST cualquiera.
- 2 Sea **v** el nodo eliminado, **actualizamos alturas y rebalanceamos** desde el padre de **v** hasta la raíz.

Desbalance a la derecha

Esto ocurre cuando $b(\mathbf{v}) = -2$, o dicho de otra forma, $b(\mathbf{v}) < -1$

Consideremos que la altura de **hi** es k y la de **hd** es $k + 2$. Los hijos izquierdo y derecho de **hd** son **wi** y **wd** respectivamente.



Desbalance a la derecha - Caso 1 (línea recta)

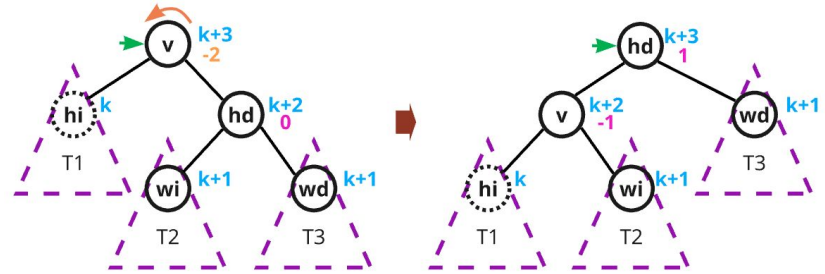
Desbalance con $b(v) = -2$

Caso 1. $b(hd) < 1$

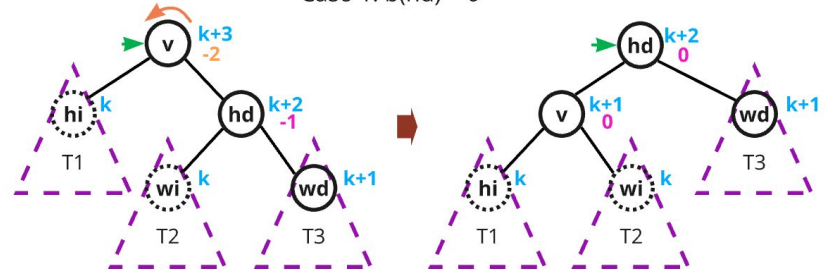
(dicho de otra manera, $b(hd) = -1$ o $b(hd) = 0$):

1 Se rota v a la izquierda.

2 El **recálculo de alturas y rebalanceo** sigue de hd a la raíz.

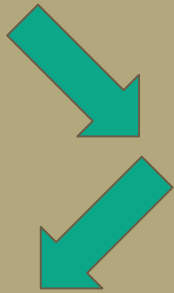


Caso 1. $b(hd) = 0$



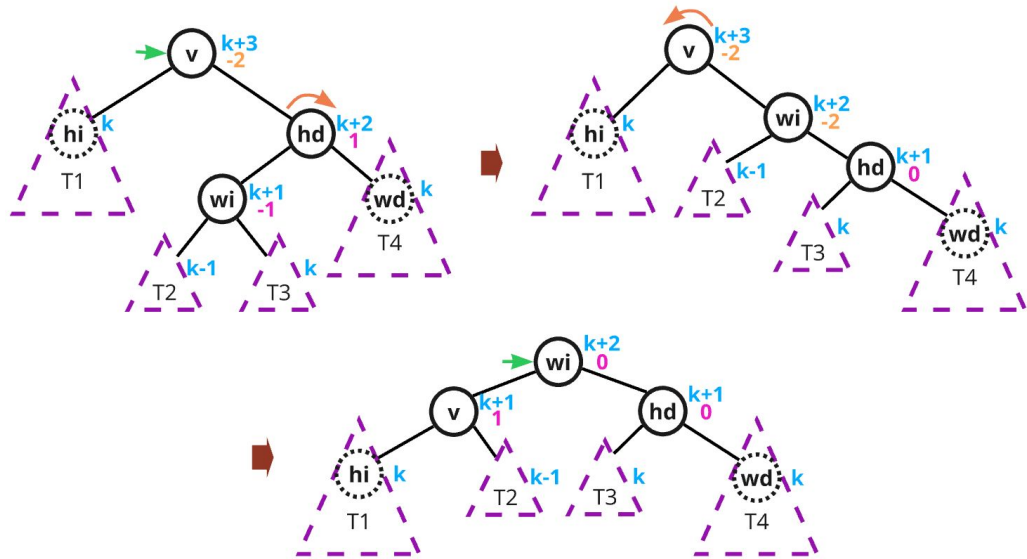
Caso 1. $b(hd) = -1$

Desbalance a la derecha - Caso 2 (zig zag)



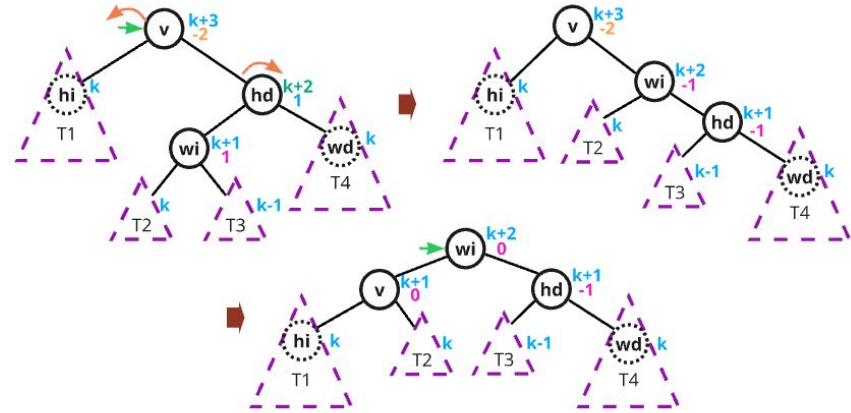
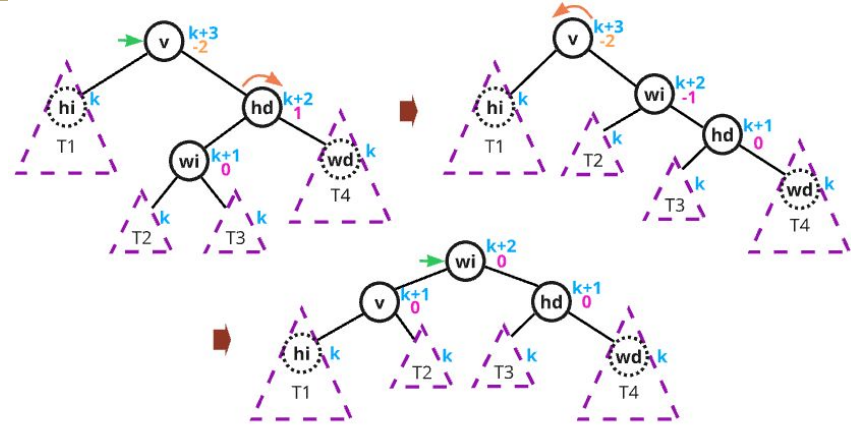
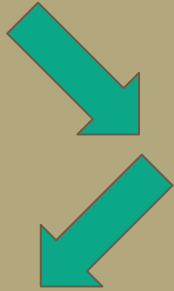
Caso 2. $b(\text{hd}) = 1$

- 1 Se rota **hd** a la derecha.
- 2 Se rota **v** a la izquierda.
- 3 El **recálculo de alturas y rebalanceo** sigue de **wi** a la raíz.



Caso 2. $b(\text{hd}) = 1$ y $b(\text{wi}) = -1$

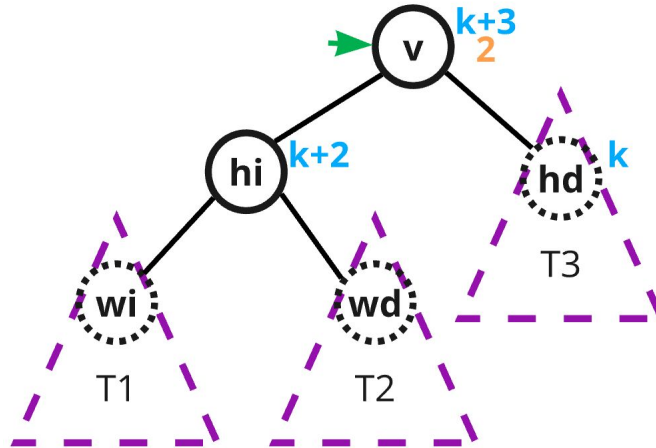
Desbalance a la derecha - Caso 2 (zig zag)



Desbalance a la izquierda

Esto ocurre cuando $b(\mathbf{v}) = 2$, o dicho de otra forma, $b(\mathbf{v}) > 1$

Consideremos que la altura de **hi** es $k + 2$ y la de **hd** es k . Los hijos izquierdo y derecho de **hi** son **wi** y **wd** respectivamente.



Desbalance a la izquierda - Caso 1 (línea recta)

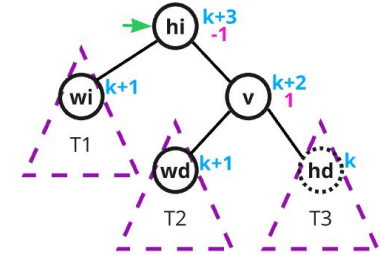
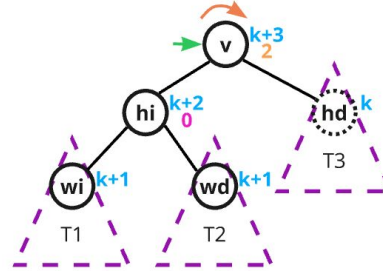
Desbalance con $b(v) = -2$

Caso 1. $b(hi) > -1$

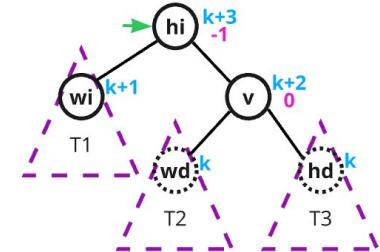
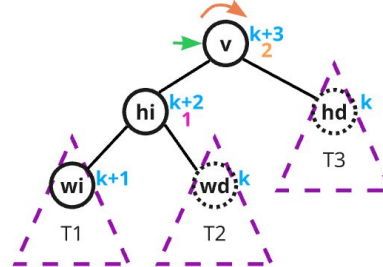
(dicho de otra manera, $b(hi) = 0$ o $b(hi) = 1$):

1 Se rota v a la derecha.

2 El **recálculo de alturas y rebalanceo** sigue de hi a la raíz.



Caso 1. $b(hi) = 0$

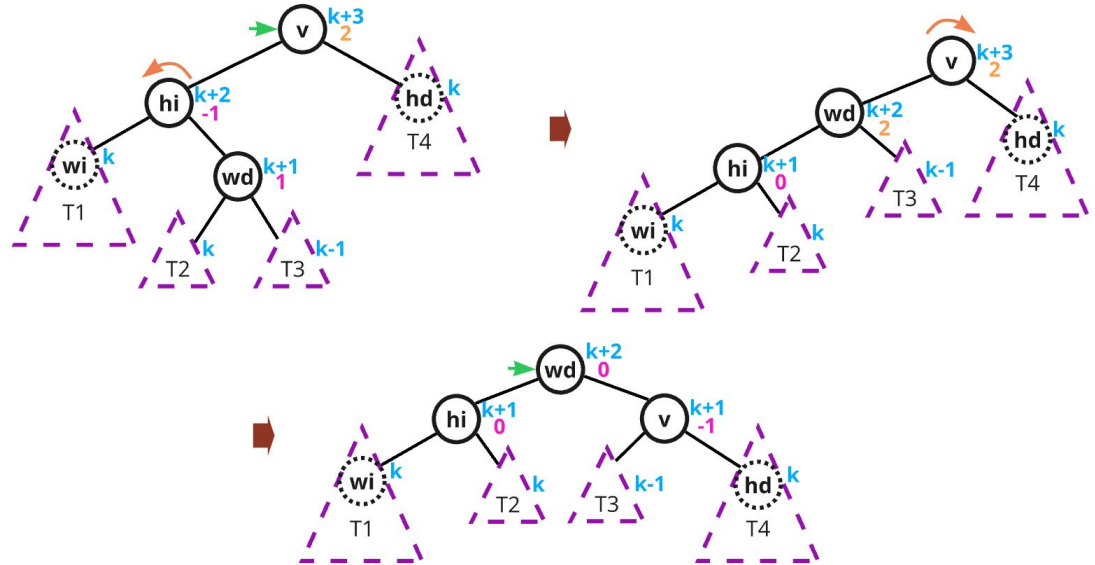


Caso 1. $b(hi) = 1$

Desbalance a la izquierda - Caso 2 (zig zag)

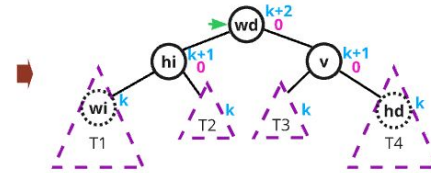
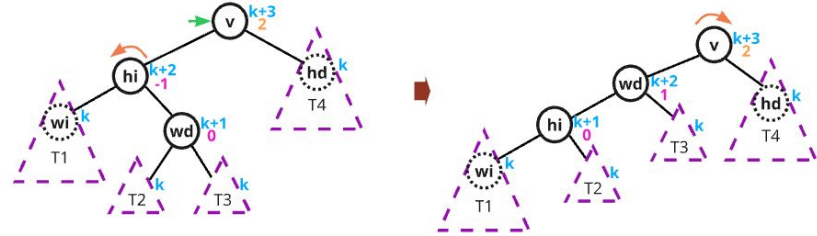
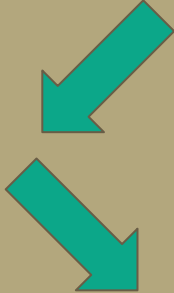
Caso 2. $b(\text{hi}) = -1$:

- 1 Se rota **hi** a la izquierda.
- 2 Se rota **v** a la derecha.
- 3 El **recálculo de alturas y rebalanceo** sigue de **wd** a la raíz.

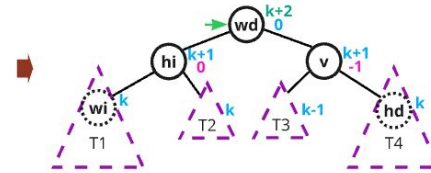
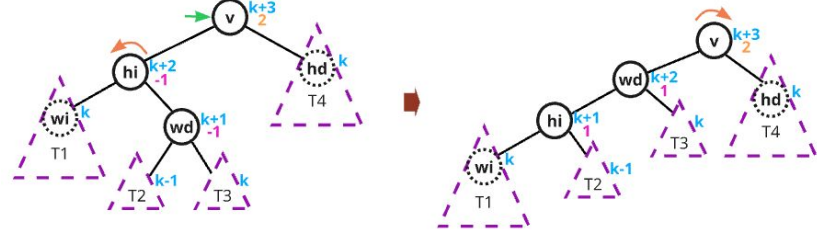


Caso 2. $b(\text{hi}) = -1$ y $b(\text{wd}) = 1$

Desbalance a la izquierda - Caso 2 (zig zag)



Caso 2. $b(hi) = -1$ y $b(wd) = 0$



Caso 2. $b(hi) = -1$ y $b(wd) = -1$

Árboles AVL

Un árbol binario de búsqueda T cumple con la propiedad de ser AVL si para todo nodo no vacío v con hijos **hi** y **hd** sus hijos izquierdo y decho respectivamente, se cumple que

$$|h(hi) - h(hd)| < 2$$

Árboles AVL

La condición anterior obliga a que todos los nodos deben estar balanceados, ya que la altura de sus hijos difiere a lo más de una unidad. Un árbol se puede desbalancear cuando:

- Se inserta un nuevo elemento
- Se elimina un nuevo elemento

Inserción en árboles AVL

1. Se inserta el elemento como en un árbol binario de búsqueda
2. Se rebalancean los vértices y se actualizan las alturas desde el nodo v hasta la raíz